

사전 훈련된 신경망

2장에서 다루는 내용

- 사전 훈련된 이미지 인식 모델 돌려보기
- GAN과 사이클GAN 소개
- 이미지에서 텍스트 설명을 만들어낼 수 있는 자막 모델
- 토치 허브에 모델 공유

2장에서 흥미로운 내용을 보여주기로 약속하며 1장을 마무리했는데, 이제 공개할 시간이 왔다. 컴퓨터 비전^{computer vision}은 딥러닝 발전으로 가장 많이 영향받은 분야 중 하나다. 이미지 내용을 분류하고 해석하기 위한 요구사항이 존재했고, 엄청난 데이터가 사용 가능해졌으며, 컨볼루션층과 같은 새로운 구조도 나왔고, 미증유의 정확도로 GPU에서 번개처럼 프로그램을 돌릴 수 있게 됐다. 이와 같은 요소들이 결합되기 시작한 동인은 수백만 사용자가 모바일 기기로 찍은 사진을 자동으로 이해하고 자신들의 플랫폼에서 관리하는 것을 바라는 인터넷 거인들의 열망 덕분이며 실로 엄청난 변화다.

지금부터는 대규모 공개 데이터셋으로 훈련시킨 재미있는 모델을 내려받아 돌려보면서 이 분야 최고 연구자들이 만든 모델을 즉시 사용하는 법을 배워보자. 단순히 보면 사전 훈련된 신경망은 주어진 입력에 대한 출력을 만드는 프로그램이다. 이 프로그램의 동작 방식은 신경망의 아키텍처와 훈련 과정에서 사용된 데이터가 결정한다. 원하는 입출력 쌍 또는 출력에서의 만족할 만한 속성 관점에서 모델이 구성된다. 이미 다 만들어진 모델을 사용하면 딥러닝 프로젝트에 바로 뛰어들 수 있다. 이는 해당 모델을 설계한 연구자의 전문성을 이끌어내는 동시에, 가중치를 훈련시키기 위한 계산 시간을 줄일 수 있기 때문이다.

이번 2장에서는 사전 훈련된 인기 있는 모델을 세 가지 다룬다. 즉 이미지를 이해하고 레이블을 달아주는 모델, 진짜 이미지로부터 새로운 이미지를 만들어내는 모델, 그리고 이미지의 내용을 문법에 맞게 영어 문장으로 설명하는 모델이다. 세 가지 모델을 대상으로, 사전 훈련된 모델을 파이토치로 읽어 실행하는 법을 배우고, 사전 훈련된 모델을 일관된 인터페이스를 통해 공통으로 사용할 수 있게 만드는 도구들이 모여 있는 파이토치 허브도 알아본다. 이런 과정에서 데이터 출처를 알아보고, 레이블^{label} 등의 용어를 정의해보며, 얼룩말 로데오도 살펴볼 것이다.

이미 다른 딥러닝 프레임워크를 써본 적이 있어서 바로 파이토치를 자세히 다뤄보고 싶다면 이 장은 건너뛰어도 좋다. 이 장에서는 기본적인 내용보다는 재미 위주로 설명하며, 특정 딥러닝 도구에 국한되지 않은 내용을 다룬다. 그렇다고 도구가 중요하지 않다는 의미는 아니다! 파이토치 말고 다른 딥러닝 프레임워크로 사전 훈련된 모델을 다뤄봤다면 이런 도구가 얼마나 강력한지 이미 잘 알 것이다. 더욱이 이미 생성적 적대 신경망^{GAN, Generative Adversarial Network}에 익숙한 독자라면 해당 부분의 설명은 넘어가도 좋다.

그러나 이 장에서는 몇 가지 중요한 기술을 흥미롭게 다루므로 꼭 읽어보길 바란다. 사전 훈련된 모델을 파이토치로 돌려보는 방법을 배우는 것은 두말할 나위 없이 유용한 기술이다. 모델이 이미 큰 데이터셋으로 훈련된 경우는 더욱 그렇다. 직접 모델을 훈련시키거나 그렇지 않거나 어찌 되었든 간에, 모델을 획득하고, 실제 데이터로 신경망을 돌리며, 출력을 시각화하고 평가하는 과정에 익숙해질 필요가 있다.

2.1 이미지를 인식하는 사전 훈련된 신경망

첫 번째 딥러닝 여정을 위해 사물을 인식하는 최첨단의 사전 훈련된 딥러닝 신경망을 둘러보자. 여러 소스 코드 저장소에는 접근 가능한 사전 훈련된 다양한 모델이 존재한다. 최근에는 연구자가 논문 발표와 함께 소스 코드를 공개하고 참조 데이터셋으로 모델을 훈련시켜 얻은 가중치까지 제공하는 경우가 많아졌다. 이와 같이 남이 훈련시킨 모델을 사용하면 매우 적은 노력으로 이미지 인식이 가능한 차세대 웹서비스를 만들 수 있다.

우리가 다룰 사전 훈련된 신경망은 이미지넷^{ImageNet} 데이터셋(<http://imagenet.stanford.edu>)의 일부로 훈련시킨 모델이다. 이미지넷은 스탠포드 대학에서 관리하는 1,400만 개의 이미지 세트다. 모든 이미지는, 대규모 영어 어휘 데이터셋인 워드넷^{WordNet} 데이터셋(<http://wordnet.princeton.edu>)으로서 계층 관계를 지니는 명사로 레이블되어 있다.

다른 공개 데이터셋과 마찬가지로 이미지넷 데이터셋도 학술 경연대회에서 유래했다. 경연대회는 전통적으로 학교나 회사 연구원들이 정기적으로 참여하는데, 그중에서도 이미지넷 이미지 인식 대회 ImageNet Large Scale Visual Recognition Challenge, 줄여서 ILSVRC로 부르는 대회가 2010년에 시작했으며 가장 큰 인기를 끌었다. 이 특별한 대회에서는 몇 가지 작업을 해결해야 하며, 주제는 매년 바뀐다. 예를 들면 이미지에 포함된 사물의 유형이 무엇인지 구분하는 이미지 분류, 이미지 내의 사물 위치를 식별하는 사물 측위, 이미지에서 사물을 파악하고 레이블링하는 사물 인식, 이미지의 상황을 분류하는 장면 분류, 이미지에서 소, 말, 치즈, 모자 등 의미 분류에 해당하는 영역을 떼어내는 장면 파싱 같은 주제가 대표적이다. 이 중 이미지 분류 작업은 주어진 이미지를 본 후 천 가지 범주 가운데 다섯 개의 레이블을 뽑아내야 하는데, 이미지 내 콘텐츠를 설명하고 신뢰도 순으로 나열해야 한다.

ILSVRC의 훈련셋은 120만 개의 이미지로, 각 이미지에 해당하는 클래스^{class}가 ‘강아지’ 같은 1,000개의 명사로 레이블링되어 있다. 앞으로는 클래스라는 말과 레이블이라는 말을 혼용해 쓸 것이다. 이미지넷 데이터의 일부는 그림 2.1과 같다.

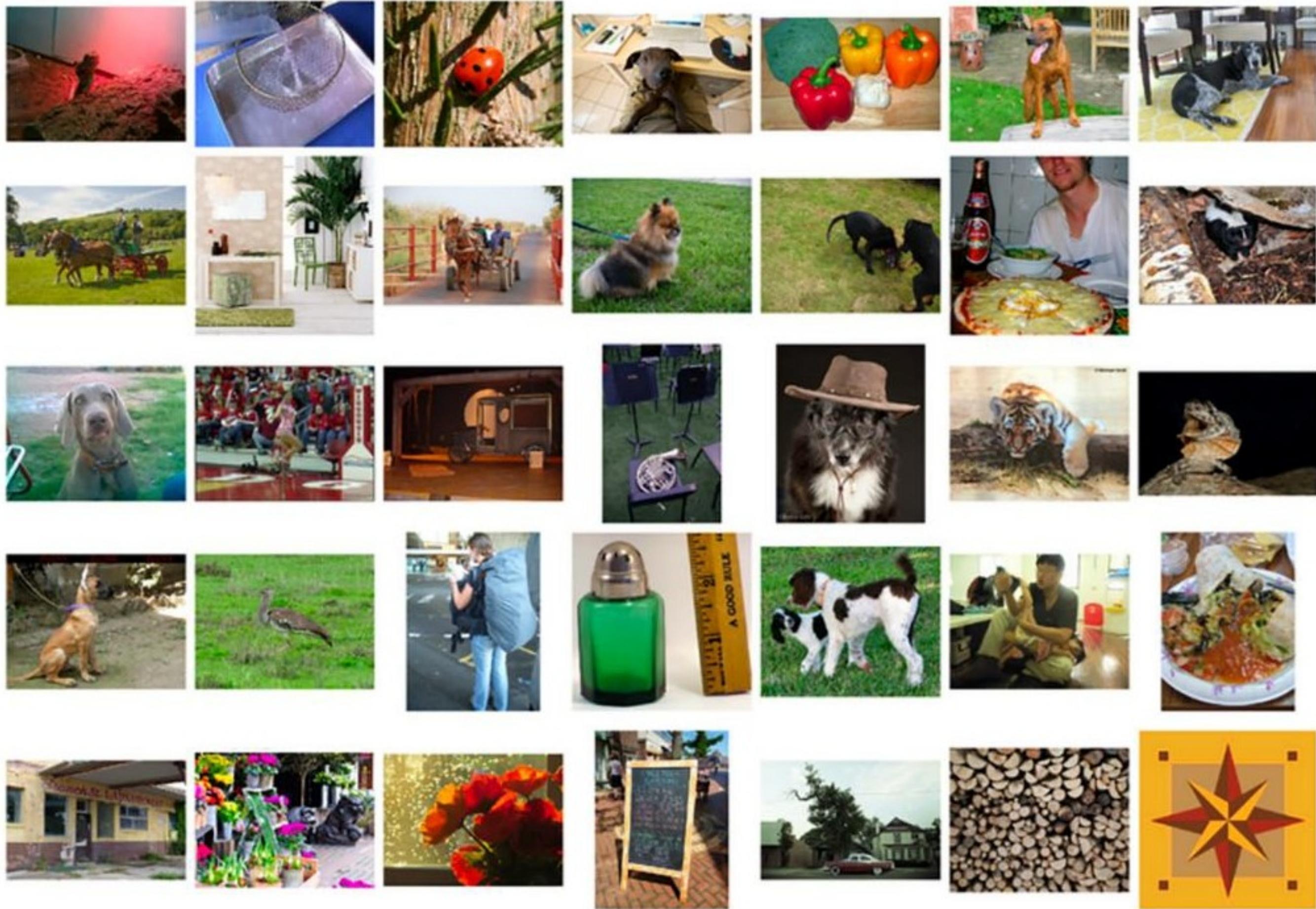


그림 2.1 이미지넷 이미지 일부 샘플

이 장의 마지막에 이르면, 그림 2.2의 강아지 그림 대신 아무 이미지나 사전 훈련된 모델에 넣어 볼 수 있다. 그러면 결과로 예측된 레이블 목록이 나오는데 이를 통해 모델이 그림을 어떻게 파악했는지 알 수 있다. 예측은 정확할 수도 혹은 그렇지 않을 수도 있다!

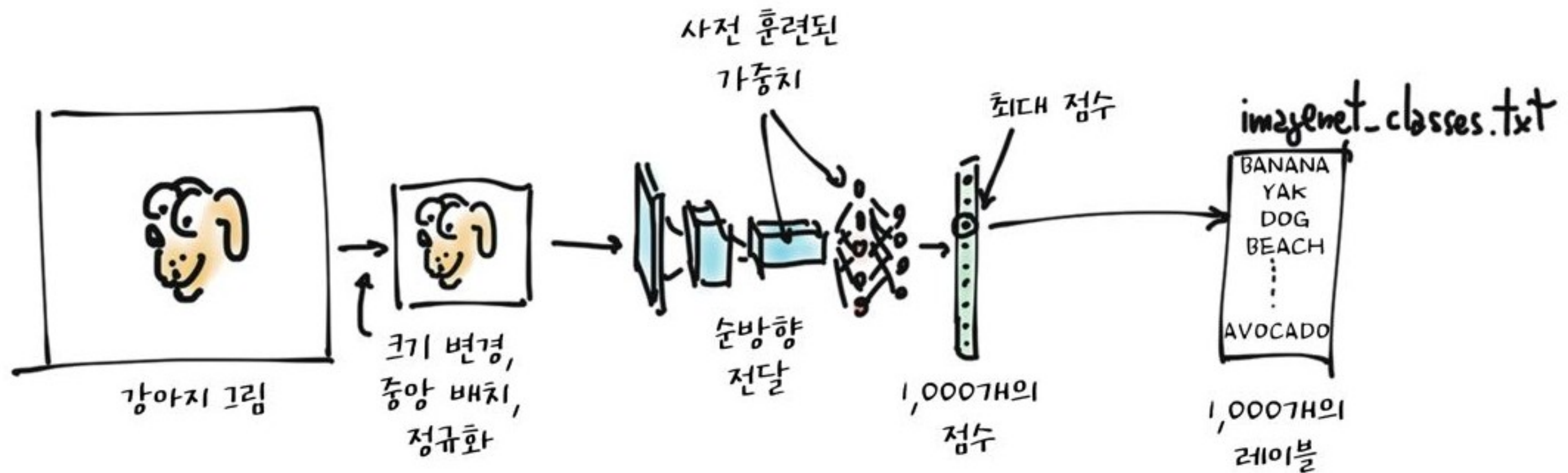


그림 2.2 추론 과정

입력 이미지는 전처리되어 다차원 배열 클래스인 `torch.Tensor`의 인스턴스로 변환된다. 높이와 너비를 가진 RGB 이미지이므로 이 텐서는 3차원으로 구성되는데 한 차원은 삼색 채널 그리고 다른 두 차원은 특정 크기의 공간으로 된 이미지 차원이 될 것이다(3장에서 텐서를 자세히 알아보겠지만, 일단은 부동소수점 수의 벡터나 행렬이라 생각해두자). 우리의 모델은 전처리된 입력 이미지를 사전 훈련된 신경망에 통과시켜 각 클래스에 대해 점수를 계산한다. 가장 높은 점수는 가중치에 따라 가장 그럴싸한 클래스에 대응한다. 각 클래스는 레이블과 일대일로 매핑되어 있다. 출력은 1,000개의 요소를 가진 `torch.Tensor`에 포함되어 있으며 각 요소는 해당 클래스와 관련된 점수를 대표한다.

2.1.1 이미지 인식을 위해 사전 훈련된 신경망 가져오기

앞서 말한 것처럼 이제 이미지넷 데이터셋으로 훈련된 신경망을 가져올 것이다. 먼저 토치비전 TorchVision 프로젝트(<https://github.com/pytorch/vision>)를 살펴보자. 여기에서는 알렉스넷 AlexNet(<http://mng.bz/lo6z>), 레즈넷 ResNet(<https://arxiv.org/pdf/1512.03385.pdf>), 인셉션 Inception v3(<https://arxiv.org/pdf/1512.00567.pdf>) 같은 컴퓨터 비전용 고성능 신경망 아키텍처를 볼 수 있다. 또한 이미지넷 같은 데이터셋에도 쉽게 접근할 수 있으며, 파이토치로 컴퓨터 비전 애플리케이션을 빠르게 만들 수 있는 유틸리티도 제공한다. 일부는 이 책에서도 좀 더 자세히 살펴볼 예정이다. 지금은 두 개의 신경망을 받아 실행해보자. 하나는 혁신적인 초기 이미지 인식 신경망 중 하나인 알렉스넷이고 다른 하나는 2015년에 이미지넷 분류, 인식, 측위에서 최고 점수를 받은 잔차 신경망 residual network, 즉 레즈넷이다. 1장에서 파이토치를 가져와서 돌려보지 않았다면 지금 당장 시도해보자.

사전 훈련된 모델은 `torchvision.models`([code/p1ch2/2_pre_trained_networks.ipynb](#))에 있다.

```
# In[1]:  
from torchvision import models
```

여러 모델을 실제로 볼 수 있다.

```
# In[2]:  
dir(models)  
  
# Out[2]:  
['AlexNet',  
 'DenseNet',  
 'Inception3',  
 'ResNet',  
 'SqueezeNet',  
 'VGG',  
 ...  
 'alexnet',  
 'densenet',  
 'densenet121',  
 ...  
 'resnet',  
 'resnet101',  
 'resnet152',  
 ...  
 ]
```


파이썬 클래스를 참조하는 대문자로 시작하는 이름이 유명 모델들의 실제 구현체다. 모델마다 아키텍처가 서로 다른데 단순히 생각하면 입력과 출력 사이에 실행하는 연산의 배치가 다르다고 보면 된다. 소문자로 시작하는 이름은 편의상 모델 클래스의 인스턴스를 바로 받아쓸 수 있는 함수이며, 때때로 파라미터 집합이 달라진다. 예를 들어 `resnet101`은 계층이 101개인 레즈넷 모델을 넘겨주고 `resnet18`은 계층이 18개인 모델을 넘겨주는 식이다. 이제 알렉스넷을 집중해서 살펴보자.

2.1.2 알렉스넷

알렉스넷^{AlexNet}은 2012년 ILSVRC에서 큰 격차로 우승했는데 정답 레이블이 출력값에서 상위 5위 클래스 안에 들어 있어야 하는 ‘상위 5위 테스트’에서 15.4%의 오차율을 보였다. 2위는 심층 신경망을 사용하지 않은 모델로, 26.2%의 오차율을 기록했다. 컴퓨터 비전 역사상 기록적인 순간이자 커뮤니티의 모든 이가 비전에 대한 딥러닝의 가능성을 깨닫기 시작한 순간이었다. 이후 지속적인 개선이 이뤄졌으며, 최근에 개선된 아키텍처와 훈련 방법을 사용할 경우 상위 5위 테스트 오차율은 3%까지 줄어들었다.

오늘날 기준에서 최신 모델과 비교하자면 알렉스넷은 소규모 신경망이라 할 수 있다. 상대적으로 작지만, 실제 동작하는 신경망을 가볍게 살펴보고 새 이미지를 사전 훈련된 모델에 넣어보기에는 최적이다.

그림 2.3에서 알렉스넷의 구조를 볼 수 있다. 완벽하게 이해할 수 있도록 자세하게 그리지는 않았지만 몇 가지 특징은 확인할 수 있다. 먼저 각 블록이 여러 개의 곱셈과 덧셈으로 이뤄져 있고 5장에서 자세히 다룰 출력에 다른 함수들을 섞는 요소도 보인다. 각각을 하나의 필터로 생각할 수 있는데, 한 개 혹은 그 이상의 이미지를 입력으로 받고 다른 형태의 이미지로 변환하여 출력하는 함수다. 중간 단계의 데이터 변환부가 각 예제와 기대값을 사용한 훈련을 통해 만들어진다.

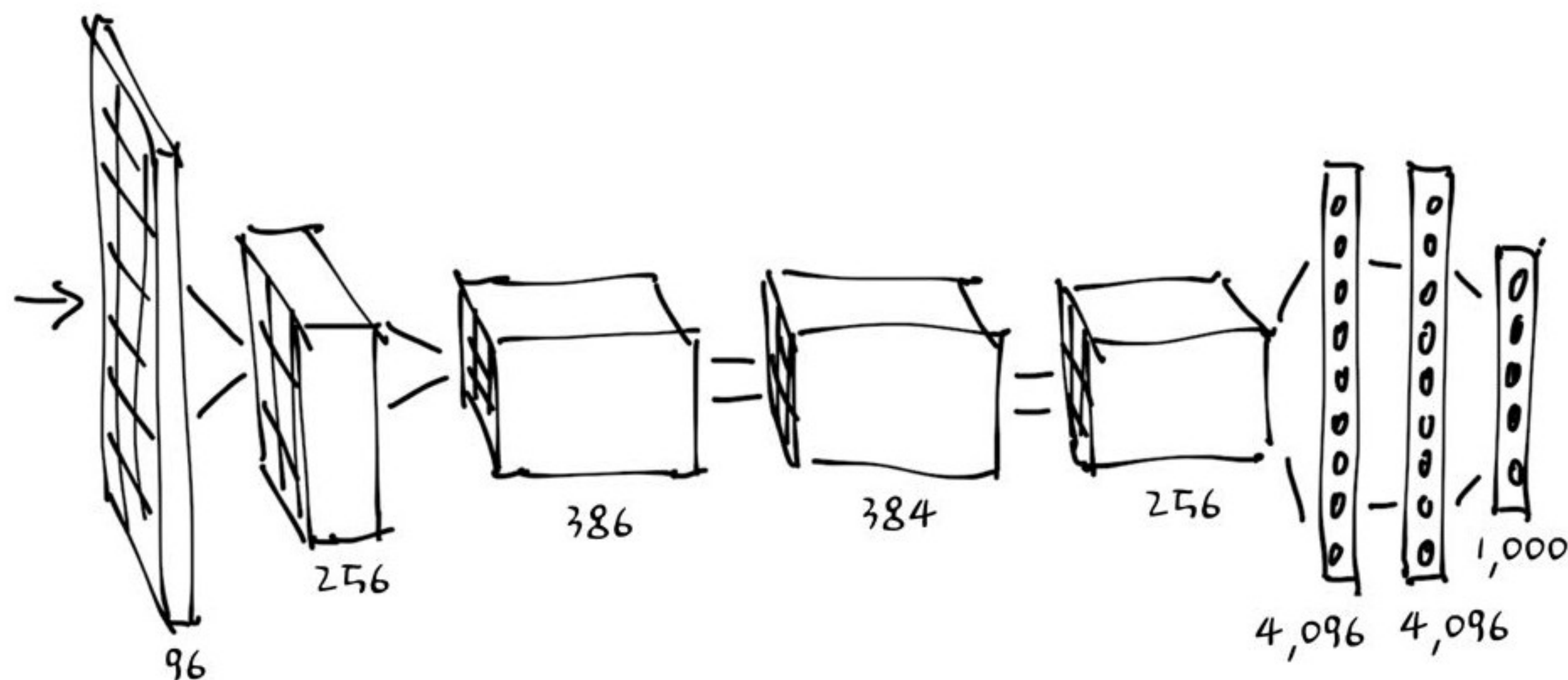


그림 2.3 알렉스넷 아키텍처

그림 2.3에서 입력 이미지가 왼쪽에서 들어오고 다섯 층의 필터를 줄줄이 거쳐가며 각 단계에서 변환된 이미지가 출력된다. 이미지들은 그림의 박스 아래에 적힌 숫자 크기로 줄어든다. 마지막 필터 다음에는 4,096개의 1차원 벡터가 있고 최종으로 1,000개의 확률값이 출력되는데 값 하나가 한 클래스에 해당한다.

알렉스넷 아키텍처에 입력 이미지를 넣어 실행하려면 먼저 다음과 같이 AlexNet 클래스로 인스턴스를 만든다.

```
# In[3]:  
alexnet = models.AlexNet()
```

여기서 alexnet은 알렉스넷 아키텍처를 실행하는 객체로, 지금 당장은 이 아키텍처의 세부 사항을 이해할 필요가 없다. 이제 AlexNet은 함수처럼 호출할 수 있는 불투명 객체가 됐다. (이미지 크기는 따로 설명할테지만) 정확한 크기의 입력 데이터를 alexnet에 넣으면, 신경망에서 순방향 전파를 수행할 수 있다. 다시 말해, 입력이 첫 번째 뉴런 집합을 거치고 다음 뉴런 집합으로 전달되면서 결국 최종 출력까지 진행된다. 구체적으로는, 적절한 타입의 input 객체가 있다면 `output = alexnet(input)`으로 순방향 전파가 가능하다는 뜻이다.

하지만 이렇게 했다가는, 입력이 전체 신경망을 지나서... 쓰레기가 나온다! 신경망이 초기화되지 않았기 때문이다. 지금은 숫자로 표현되는 가중치, 즉 입력값과 더해지거나 곱해질 파라미터들이 전혀 훈련되지 않은 백지 상태 혹은 임의의 값이 들어있는 상태다. 아예 처음부터 훈련시키거나 기존의 훈련된 값을 불러와야 한다. 지금부터 이런 작업을 진행할 것이다. 값을 불러와 보자.

먼저 `models` 모듈로 가보자. 대문자로 시작하는 이름은 컴퓨터 비전을 위한 인기 있는 아키텍처를 구현한 클래스에 대응한다고 배웠다. 소문자로 시작하는 이름은 미리 정해진 계층과 유닛의 수를 적용해 모델을 인스턴스로 만들고 옵션에 따라 미리 훈련된 가중치를 로딩한다. 이 함수를 사용하기 위해 필요한 다른 작업은 없다. 그저, 사전 훈련된 신경망이 만들어진 방식과 동일한 계층과 유닛 수를 가진 모델을 편리하게 인스턴스로 만들어낼 뿐이다.

2.1.3 레즈넷

다음으로 resnet101 함수를 사용해서 계층 수가 101개인 컨볼루션 신경망을 만들어볼 것이다. 조금 넓게 바라보자면, 2015년 잔차 신경망^{residual network}이 만들어지기 전까지 이 정도의 깊은 신경망을 안정적으로 훈련시키는 것은 매우 어려웠다. 잔차 신경망을 약간의 트릭으로 이를 가능하게 만들었고, 그 해에는 다른 여러 벤치마크 결과를 압도했다.

레즈넷 신경망의 인스턴스를 한번 만들어보자. 함수에 인자를 넣어 1,000개의 카테고리로 구분한 120만 개의 이미지 데이터셋인 이미지넷^{ImageNet}으로 훈련시킨 resnet101의 가중치를 내려받자.

```
# In[4]:  
resnet = models.resnet101(pretrained=True)
```

다운로드부터 모델을 생성하는 데까지 1분이 넘게 걸릴 수도 있다. 가만히 있어도 4,450만 개나 되는 파라미터를 최적화된 값으로 세팅해주니 충분히 기다릴 만하다!

2.1.4 준비, 시...작 전에 잠깐만

방금 뭘 받았더라? 궁금하니 resnet101이 어떻게 생겼나 한번 살펴보자. 내려받은 모델의 값을 출력해볼 수 있다. 그림 2.3에서 본 것과 동일한 종류의 텍스트 표현을 확인할 수 있다. 출력 결과를 보면 신경망 구조의 세부를 알 수 있다. 지금 당장은 정보 과잉처럼 느껴지지만, 책을 읽어 갈수록 이 짧은 코드가 우리에게 의미하는 바를 더 잘 이해하게 될 것이다.

```
# In[5]:
```

```
resnet
```

```
# Out[5]:
```

```
ResNet(  
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)  
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
  (relu): ReLU(inplace)  
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)  
  (layer1): Sequential(  
    (0): Bottleneck(  
...  
    )  
  )  
  (avgpool): AvgPool2d(kernel_size=7, stride=1, padding=0)  
  (fc): Linear(in_features=2048, out_features=1000, bias=True)  
)
```


코드의 각 줄마다 `modules`를 사용하고 있음을 볼 수 있다. 파이썬의 모듈 개념과는 무관하게 각각은 개별 연산이며, 이는 신경망을 구성하는 블록이다. 다른 딥러닝 프레임워크에서는 이를 계층^{층layer} 혹은 층이라고 부른다.

조금 더 아래로 내려가면 Bottleneck 모듈이 컨볼루션 혹은 다른 모듈을 포함하며 101번이나 반복되는 걸 볼 수 있다. 컴퓨터 비전용 심층 신경망의 전형적인 내부 구조로 필터와 비선형 함수가 어느 정도 연속적으로 나타나며 마지막에 1,000개의 출력 클래스별 점수값(`out_features`)을 만드는 `fc` 층이 붙어 있다.

`resnet` 변수는 함수처럼 사용할 수 있는데 하나 이상의 이미지를 입력으로 받아 1,000개의 이미지넷 클래스별 점수를 만든다. 이미지는 먼저 전처리가 필요한데 동일한 숫자 범위 안에 색상 값이 들어올 수 있도록 크기 조정이 필요하다. 기본적인 전처리 함수로 빠르게 파이프라인을 만들 수 있도록 `torchvision` 모듈에서 `transforms`를 제공한다.

```
# In[6]:  
from torchvision import transforms  
preprocess = transforms.Compose([  
    transforms.Resize(256),  
    transforms.CenterCrop(224),  
    transforms.ToTensor(),  
    transforms.Normalize(  
        mean=[0.485, 0.456, 0.406],  
        std=[0.229, 0.224, 0.225]  
    )])
```

예제 코드는 입력 이미지의 크기를 256×256 으로 조정하고 중심으로부터 224×224 로 잘라낸 후 파이토치 다차원 배열인 텐서 형태로 전환한다. 이 예에서는 색, 높이, 너비의 3차원 배열인 텐서로 전환한다. 그리고 지정된 평균과 표준편차를 가지도록 RGB(적, 녹, 청)를 정규화한다. 훈련에 사용하는 이미지 형식과 일치하게 만들기 위해 이런 작업이 필요하며 이렇게 해야 신경망이 의미있는 답을 제공한다. 7.1.3절에서는 이미지 인식 모델을 직접 만들 때 이미지 변환에 대해 좀 더 구체적으로 알아볼 것이다.

이제 깃허브 저장소에서 bobby.jpg 같은 강아지 사진을 하나 골라 전처리 작업을 수행하고 나서 레즈넷이 어떻게 판단하는지 확인할 수 있다. 파이썬 이미지 조작 모듈인 필로우 Pillow(<https://pillow.readthedocs.io/en/stable>)를 사용하면 로컬 파일시스템에 있는 이미지를 바로 불러올 수 있다.


```
# In[7]:  
from PIL import Image  
img = Image.open("../data/p1ch2/bobby.jpg")
```

주피터 노트북으로 따라 하는 중이라면 화면에서 바로 사진을 볼 수 있다(다음 <PIL.JpegImagePlugin...에 해당하는 부분에 보일 것이다).

```
# In[8]:  
img  
  
# Out[8]:  
<PIL.JpegImagePlugin.JpegImageFile image mode=RGB size=1280x720 at 0x1B1601360B8>
```

아니라면 다음과 같이 show 함수를 실행해서 그림 2.4처럼 새 창으로 뷰어를 띄울 수 있다.

```
>>> img.show()
```



그림 2.4 우리의 특별한 강아지 바비 이미지를 입력으로 사용해본다.

이제 만들어놓은 전처리 파이프라인으로 이미지를 통과시키자.

```
# In[9]:  
img_t = preprocess(img)
```

이렇게 만들어진 텐서를 리셰이프하고 잘라낸 후 정규화해서 신경망에 넣을 수 있도록 준비한다. 이 과정은 3장과 4장에서 자세히 살펴볼 테니 지금은 그냥 넘어가자.

```
# In[10]:  
import torch  
batch_t = torch.unsqueeze(img_t, 0)
```

이제 모델을 실행할 준비가 됐다.

2.1.5 실행!

딥러닝 사이클에서 훈련된 모델에 새로운 데이터를 넣어 결과를 보는 과정을 **추론**^{inference}이라고 한다. 추론을 수행하려면 신경망을 eval 모드로 설정해야 한다.

```
# In[11]:
resnet.eval()

# Out[11]:
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, cell_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
  ...
    )
  )
  (avgpool): AvgPool2d(kernel_size=7, stride=1, padding=0)
  (fc): Linear(in_features=2048, out_features=1000, bias=True)
)
```

깜박 잊고 모드 설정을 빠뜨린다면, 배치 정규화^{batch normalization}나 드롭아웃^{dropout} 같은 몇몇 사전 학습된 모델은 이상한 결과를 만들어낸다. eval 모드가 설정됐으니 이제 추론을 수행해보자.

```
# In[12]:
out = resnet(batch_t)
out

# Out[12]:
tensor([[ -3.4803, -1.6618, -2.4515, -3.2662, -3.2466, -1.3611,
          -2.0465, -2.5112, -1.3043, -2.8900, -1.6862, -1.3055,
          ...,
          2.8674, -3.7442,  1.5085, -3.2500, -2.4894, -0.3354,
          0.1286, -1.1355,  3.3969,  4.4584]])
```

4,450만 개에 이르는 파라미터가 관련된 엄청난 연산이 방금 실행되어 1,000개의 스코어를 만들어냈고 점수 하나 하나는 이미지넷 클래스에 각각 대응된다. 시간도 얼마 걸리지 않았다.

이제 점수가 가장 높은 클래스의 레이블만 찾으면 된다. 레이블은 모델이 이미지에서 무엇을 봤는지를 우리에게 알려준다. 사람이 인식한 것과 같은 레이블이라면 훌륭하다! 모든 게 잘 동작한다. 혹 아니라면 훈련 중에 뭔가가 잘못됐거나 입력으로 넣은 이미지가 모델이 예상하는 바와 너무 달라서 적당하게 처리할 수 없는 상황이거나, 엇비슷한 다른 문제 때문일 것이다.

예측한 순서대로 레이블 리스트를 만들기 위해서는, 출력과 동일한 순서로 나열된 레이블 텍스트 파일을 읽어놓은 후 점수가 높은 출력의 인덱스부터 동일한 인덱스에 있는 레이블을 리스트에 추가하면 된다. 이미지 인식 관련 모델 대부분은 여기서 보여주는 출력 형태와 유사하다.

이미지넷 데이터 클래스에 대한 1,000개의 레이블이 담긴 파일을 읽어보자.


```
# In[13]:  
with open('../data/p1ch2/imagenet_classes.txt') as f:  
    labels = [line.strip() for line in f.readlines()]
```

이 시점에서는 이전 과정에서 얻은 out 텐서에서 가장 높은 점수에 대응하는 색인을 찾아야 한다. 파이토치의 max 함수를 사용해 텐서에서 최댓값과 이 최댓값이 들어 있는 부분의 인덱스를 출력하자.

```
# In[14]:  
_, index = torch.max(out, 1)
```

다음으로 레이블에 접근하기 위해 직전에 출력한 인덱스를 사용할 수 있다. 여기서는 인덱스가 단순한 파이썬 숫자가 아닌 1개 요소를 가지는 1차원 텐서(tensor([207]))다. 그래서 labels 리스트의 인덱스로 사용하기를 원하는 실제 정수 값을 얻기 위해 index[0]을 사용할 필요가 있다. 그리고 출력을 [0, 1] 사이의 값으로 정규화하고 전체 합으로 나누기 위해 torch.nn.functional.softmax(<http://mng.bz/BYnq>)를 사용한다. 이렇게 해서 나온 값은 모델의 예측에 대한 신뢰도와 거의 비슷하다고 볼 수 있다. 따라서 이 예제에서 모델은 입력 이미지가 골든리트리버일 확률이 96%라고 확신한다.

```
# In[15]:
percentage = torch.nn.functional.softmax(out, dim=1)[0] * 100
labels[index[0]], percentage[index[0]].item()

# Out[15]:
('golden retriever', 96.29334259033203)
```

우쭈쭈, 우리 착한 아기 어디 있지?

모델은 여러 점수를 출력하므로, 두 번째, 세 번째 등의 결과가 무엇인지도 찾아낼 수 있다. 이를 위해 `sort` 함수를 사용할 수 있으며, 배열에서 값을 기준으로 오름차순이나 내림차순으로 정렬한 후 값과 인덱스를 보여준다.

```
# In[16]:
```

```
_, indices = torch.sort(out, descending=True)  
[(labels[idx], percentage[idx].item()) for idx in indices[0][:5]]
```

```
# Out[16]:
```

```
[('golden retriever', 96.29334259033203),  
 ('Labrador retriever', 2.80812406539917),  
 ('cocker spaniel, English cocker spaniel, cocker', 0.28267428278923035),  
 ('redbone', 0.2086310237646103),  
 ('tennis ball', 0.11621569097042084)]
```


처음 나오는 네 가지 결과는 강아지다(레드본이라는 종은 처음 본다). 이후로는 점점 재미있어지는데, 다섯 번째 결과는 테니스공이다. 강아지와 테니스공이 같이 있는 이미지가 무척 많기 때문에, 아마도 모델은 “테니스공 모양을 착각할 확률은 0.1% 정도”라고 말하는 것 같다. 이런 실험 결과를 통해 인간과 신경망이 사물을 바라보는 근본적인 관점의 차이를 알 수 있으며, 동시에 이상하거나 기이한 값을 데이터에 살짝 끼워넣기가 얼마나 쉬운지도 알 수 있다.

이제부터는 재밌게 즐겨보자! 아무 이미지나 가져다가 모델을 돌려보고 어떤 결과가 나오는지 한번 살펴보자. 신경망이 얼마나 성공적으로 판단하는지는 학습셋이 분류에 맞춰 얼마나 잘 표현되었는지에 크게 영향을 받는다. 훈련셋에 없는 분류의 이미지를 보여주면 높은 확률로 엉뚱한 답을 제시하기도 한다. 본 적 없는 데이터에 대해 모델이 어떻게 반응하는지 느껴보고 실험해보는 것은 무척 유용하다.

지금까지 2015년 이미지 분류 대회에서 우승한 신경망을 하나 돌려봤다. 이 모델은 강아지 예제로부터 강아지를 식별하도록 학습했고 수많은 다른 사물 분류에 대해서도 학습했다. 이제, 다른 모델은 또 그 밖에 어떤 일을 할 수 있는지, 이미지 생성 모델부터 차근차근히 살펴보자.

2.2 가짜 이미지를 만드는 사전 훈련된 모델

우리가 유명한 예술가의 잃어버린 작품에 대해 위조품을 만들어 파는 사기꾼이라 생각해보자. 우리는 범죄자이지 화가가 아니므로, 렘브란트나 피카소의 그림을 모사해 그리면 한눈에 아마추어가 그린 모조품인 게 들통날 것이다. 아무리 오랜 시간을 쏟고 연습을 거듭해 정말 진짜처럼 그려본대도, 지역 예술 경매에서조차 바로 쫓겨날 것이다. “이거 가짜네. 당장 나가요.” 얘기를 들어 봐야 별 도움이 안될 테니 일단은 여러 종류의 위작을 만들고 그나마 가짜인 게 들통나기 어려운 것을 좀 더 발전시켜 구별하기 힘들게 해야 할 텐데, 그러려면 시간이 너무 오래 걸린다.

아니면 도덕성에 의심이 가는 미술사가를 불러다가 우리의 작품을 살펴보고 어떤 점이 가짜로 보이는지 귀뜸해달라는 편이 나을 테다. 그 전문가들의 피드백을 받아서 명확하게 방향을 잡아 결과물을 계속 개선해나가면, 학자들은 결국 우리 그림과 진짜 그림을 구분해내지 못할 것이다.

드디어 우리는 루브르 박물관의 ‘보티첼리’ 작품을 손에 넣어서 그들의 돈을 우리 호주머니에
쏟아 담아 곧 부자가 될 것이다!

다소 우스꽝스럽게 얘기했지만 이에 대한 기반 기술은 타당하며, 향후 몇 년 내 디지털 데이터
의 진위 파악에 큰 영향을 미칠 수도 있다. ‘사진 증거 자료’라는 개념은 앞으로 정말 그럴듯한
가짜 사진이나 비디오를 얼마나 쉽게 자동화할 수 있느냐에 따라 흔들릴지도 모른다.

2.2.1 GAN 게임

방금 설명한 내용은 딥러닝 분야에서는 GAN 게임으로 알려져 있다. 두 개의 신경망에서 하나는 화가 역할, 다른 하나는 미술사가 역할을 하여 최고의 화가와 최고의 위작 감별사가 되기 위해 서로 경쟁한다. GAN은 **생성적 적대 신경망** Generative Adversarial Network의 약자로 용어의 순서대로 (가짜 작품이) 만들어지고 generative, 서로 더 뛰어나기 위해 경쟁하는 adversarial 신경망 network이라는 의미다. GAN은 최근 딥러닝 연구 중에서 가장 독창적인 결과물 중 하나다.

우리의 중요한 목표는 원본과 구별할 수 없는 합성된 그림을 만들어 내는 것이다. 이 위작이 합법적인 진짜 작품과 섞여 있을 때 경험 많은 전문가도 어떤 그림이 진짜인지 구별하기 어려울 것이다.

우리 시나리오에서 화가 역할을 하는 것이 생성자(generator) 신경망으로, 임의의 입력에 대해 사실 같은 이미지를 만들어내는 작업을 수행한다. 식별자(discriminator) 신경망은 도덕성을 갖추지 않은 미술 작품 감별사가 되어 생성자가 조작한 이미지인지 실제 이미지인지를 말해주는 역할이다. 대부분의 딥러닝 아키텍처와 비교할 때 이례적인 두 개의 신경망을 사용한 이 디자인은 GAN 게임에서, 믿을 수 없는 높은 수준의 결과를 만들어 낸다.

그림 2.5는 동작 원리를 개략적으로 보여준다. 생성자의 궁극적인 목표는 진짜와 가짜 이미지를 구분하지 못하게 식별자를 속이는 것이다. 맨 처음, 생성자는 렘브란트 초상화와는 전혀 닮지 않은 눈이 세 개 달린 뽀빠리 괴물을 하나 만든다. 식별자는 진짜 그림과 이 엉망진창인 그림을 쉽게 구분할 수 있다. 훈련 과정이 진행됨에 따라 식별자로부터 얻은 정보로 생성자는 점점 개선된다. 훈련이 끝날 즈음이면 생성자는 그럴싸한 가짜 이미지를 만들며, 식별자는 더 이상 진짜와 가짜를 구별할 수 없다.

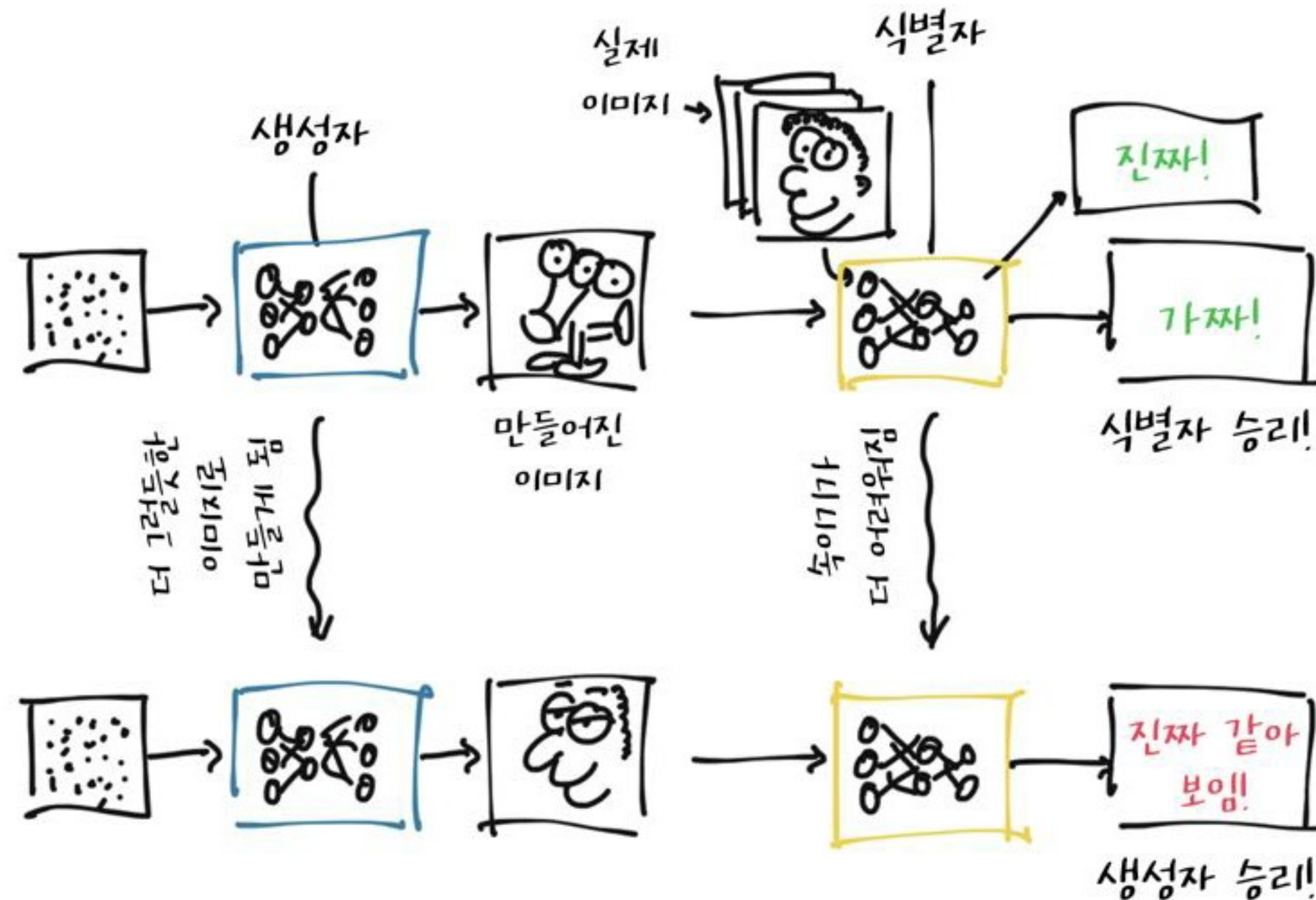


그림 2.5 GAN 게임 개념

‘식별자 승리’ 혹은 ‘생성자 승리’ 같은 명시적인 대결은 없다. 다만 두 신경망은 각각 상대의 출력값을 기준으로 신경망을 훈련시켜 각 신경망의 파라미터를 최적화한다.

이 기술은 생성자로 하여금 노이즈 값과 속성(예를 들어 얼굴이라면 안경을 쓴, 젊은, 여성이라든가) 등의 조건 시그널만으로 진짜 같은 이미지를 만들어낸다. 잘 훈련된 생성자는 심지어 사람이 보기에 진짜 같은 이미지를 생성하는 그럴듯한 모델을 학습해낸다.

2.2.2 사이클GAN

GAN 개념을 더 흥미롭게 발전시킨 것으로 사이클GAN^{CycleGAN}이 있다. 사이클GAN은 훈련셋에서 매칭된 쌍을 제공하지 않아도 한 도메인에 대한 이미지를 다른 도메인으로 바꿔준다(반대로도 가능하다). 그림 2.6은 말 사진을 얼룩말로 바꾸거나 혹은 반대로 바꾸는 작업에 대한 사이클GAN 워크플로를 나타낸다. 두 개의 개별 생성자 신경망이 있고 마찬가지로 두 개의 식별자 신경망도 있다.

그림에서 보듯이, 첫 번째 생성자는 다른 이미지(말 이미지) 분포 값을 가진 그림에서 시작하여 목표에 해당하는 얼룩말 이미지의 분포 값과 일치하는 이미지를 만들도록 학습한다. 그래서 말 사진으로부터 만들어진 이미지가 진짜 얼룩말 사진인지 아닌지를 식별자가 구별하지 못하게 한다. 동시에 만들어진 가짜 얼룩말 사진이 다른 생성자로 전달되어 맞은편의 식별자가 판별하도록 보내는 사이클이 있다(그래서 사이클GAN이라 불린다). 이렇게 사이클을 만듦으로써 훈련 과정에서 문제가 되었던 GAN의 안정화 이슈를 해결할 수 있다.

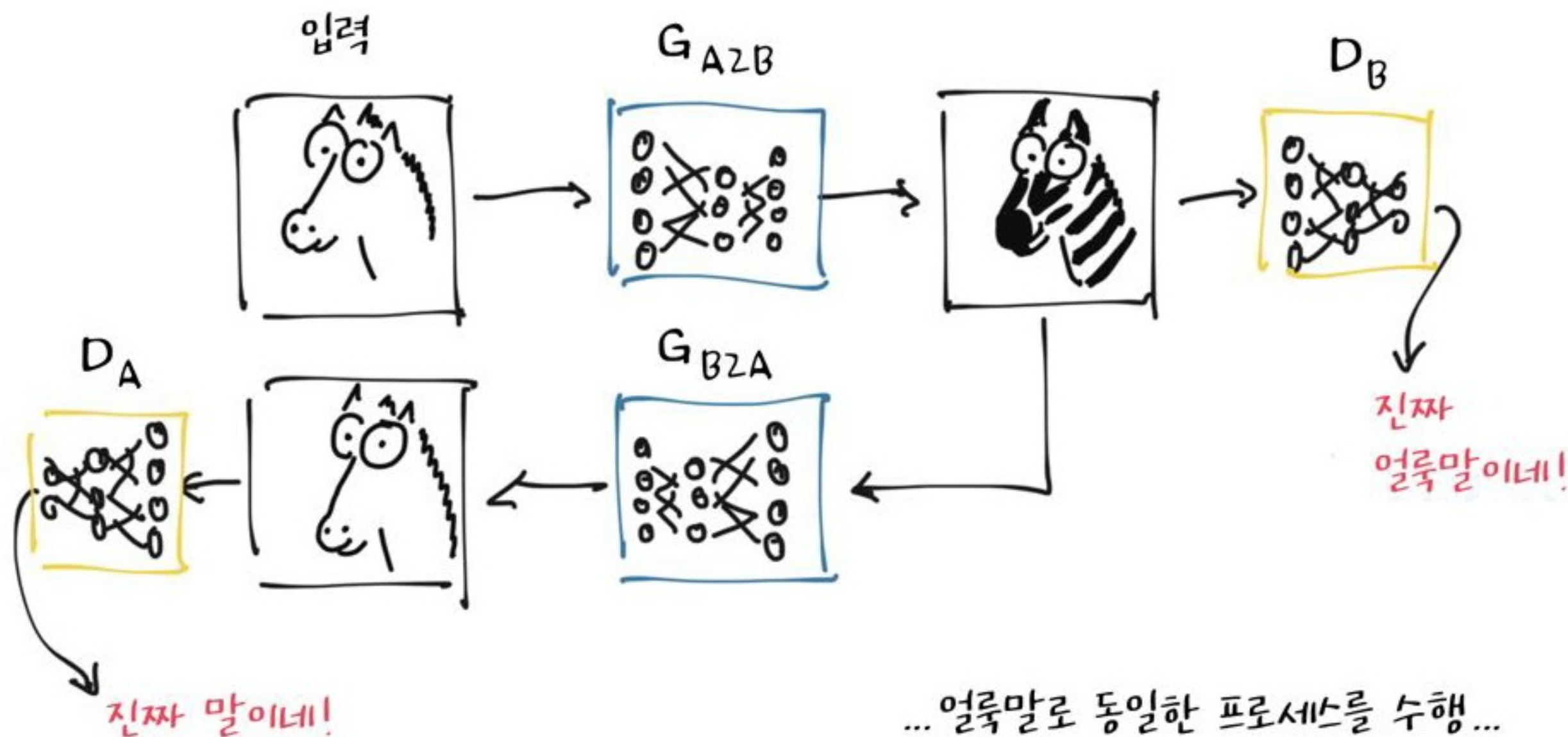


그림 2.6 두 식별자 신경망을 모두 속일 때까지 훈련하는 사이클GAN

각도가 동일한 말과 얼룩말 사진이 필요하지 않다는 점이 상당히 재미있다(동일한 포즈를 취한 사진을 찍으려면 운이 좋아야 한다!). 서로 관련 없는 말과 얼룩말 이미지 모음만으로 충분히 생성자가 학습을 시작할 수 있으며, 이는 지도 학습 설정(의 불편함)을 뛰어넘는다. 뿐만 아니라 생성자는 뭘 어떻게 해야 하는지 알려주지 않아도 사진 속 사물의 모습을 어떻게 바꿔야 하는지 배운다. 어떤 게 갈기인지 다리인지 알려주는 시그널도 없지만 다른 동물의 구조에 맞춰 변환 작업을 진행한다.

2.2.3 말을 얼룩말로 바꾸는 신경망

지금 바로 이 모델을 사용해 볼 수 있다. 사이클GAN 신경망은 이미지넷 데이터셋에 있는 (서로 관련 없는) 말과 얼룩말 이미지로 훈련된 상태다. 신경망은 이미지의 다른 부분은 최대한 건드리지 않으면서 말 사진을 얼룩말로 바꾸도록 학습한다. 말을 얼룩말로 바꾸는 도구는 인류가 수천 년 동안 숨죽여 기다려왔으며, 이런 변환 작업은 학습에 관여하지 않고서도 복잡한 실세계 프로세스를 모델링하는 사이클GAN의 능력을 보여준다. 물론 GAN 아키텍처가 아직은 제한적이지만, 가까운 미래에는 사람이 영상의 진위 여부를 판별하기 어려워질지도 모른다. 이는 판도라의 상자를 여는 순간일지도 모른다.

사전 훈련된 사이클GAN을 사용해 보면 생성자가 어떻게 구현되었는지 자세히 살펴볼 수 있을 것이다. 우리의 친구 레즈넷을 사용할 것이다. ResNetGenerator 클래스는 여기서 정의하지 않는다. ResNetGenerator 클래스와 관련된 코드는 3_cyclegan.ipynb의 첫번째 셀에 위치하며, 구현은 지금 당장은 중요하지 않은데, 파이토치 경험을 충분히 쌓기 전에는 이해하기가 좀 난해할 것이다. 일단은 ‘어떻게’ 동작하는지보다는 이걸로 ‘뭘’ 할 수 있는지에 집중해보자. 기본 파라미터로 클래스 인스턴스를 만들어보자(`code/p1ch2/3_cyclegan.ipynb`).

```
# In[2]:
```

```
netG = ResNetGenerator()
```

netG 모델을 만들었지만 내부 가중치가 임의로 들어 있다. 생성자 모델이 이미 horse2zebra 데이터셋을 사용해 사전 훈련되었다고 언급했었는데, 1,068개의 말 사진과 1,335개의 얼룩말 사진이 훈련셋에 들어있다. 데이터셋은 <http://mng.bz/8pKP>에 있다. 모델 가중치는 .pth 파일에 저장되어 있는데 모델의 텐서 파라미터를 pickle 포맷으로 저장했을 뿐이다. 이 가중치를 ResNetGenerator에 로딩할 때 load_state_dict 함수를 사용한다.

```
# In[3]:
```

```
model_path = '../data/plch2/horse2zebra_0.4.0.pth'
```

```
model_data = torch.load(model_path)
```

```
netG.load_state_dict(model_data)
```

이제 netG가 훈련을 통해 얻은 모든 지식을 흡수했다. 2.1.3절에서 torch-vision으로 resnet101에 로딩했을 때와 동일하지만 앞에서는 torchvision.resnet101 함수 안에서 실행되어 로딩 과정이 보이지 않았다.

resnet101에서 했던 것처럼 신경망을 eval 모드로 설정하자.

```
# In[4]:  
netG.eval()  
  
# Out[4]:  
ResNetGenerator(  
  (model): Sequential(  
    ...  
  )  
)
```


전에 했던 것처럼 모델을 출력해보자. 모델이 하는 작업을 생각해보면 복잡한 구조가 충분히 이해된다. 이미지를 픽셀 단위로 보고 한 개 이상의 말을 찾은 다음, 각 픽셀을 수정해서 실제 같은 얼룩말을 만드는 작업이니까. 출력된 모델이나 소스 코드에 얼룩말 관련 정보는 찾아볼 수 없다. 신경망은 뼈대고 가중치는 살이라고 생각해보자.

이제 아무 이미지나 넣고 생성자가 만든 결과를 볼 준비가 됐다. PIL과 torchvision을 임포트해야 한다.

```
# In[5]:  
from PIL import Image  
from torchvision import transforms
```

이제 신경망에 넣는 데이터가 정확한 형태와 크기를 가지도록 입력을 변환하는 함수를 정의하자.

```
# In[6]:  
preprocess = transforms.Compose([transforms.Resize(256),  
                                transforms.ToTensor()])
```

말 사진을 열어보자(그림 2.7 참고).

```
# In[7]:  
img = Image.open("../data/p1ch2/horse.jpg")  
img
```



그림 2.7 말에 올라탄 사람. 말은 거부하고 있다.

음, 말 등에 올라탄 녀석이 보인다. (사진을 보건대, 그리 오래 버티지는 못할 것 같다.) 아무튼 전차를 통과시켜 적절히 형태를 맞추는 변수로 바꾸자.

```
# In[8]:
img_t = preprocess(img)
batch_t = torch.unsqueeze(img_t, 0)
```

다소 이해가지 않는 부분은 일단 걱정하지 말자. 좀 거리를 두고 이해한다는 관점으로 생각하기 바란다. 이제 모델에 batch_t를 전달할 수 있다.

```
# In[9]:
batch_out = netG(batch_t)
```

batch_out이 생성자의 출력이니, 이것을 거꾸로 이미지로 변환해보자.

```
# In[10]:
out_t = (batch_out.data.squeeze() + 1.0) / 2.0
out_img = transforms.ToPILImage()(out_t)
# out_img.save('../data/p1ch2/zebra.jpg')
out_img

# Out[10]:
<PIL.Image.Image image mode=RGB size=316x256 at 0x23B24634F98>
```

어이쿠 이런. 누가 얼룩말을 저렇게 타나? 그림 2.8의 이미지는 완벽해 보이지는 않지만 말 등에 (아무튼 간에) 올라탄 사람을 신경망이 판별하기란 쉽지 않다는 점을 고려해야 한다. 수만 장의 말 사진에 사람이 어느 영역에 있는지 윤곽을 그려주거나 일일이 포토샵으로 얼룩말을 편집해서 지도 학습에 필요한 데이터를 만들지 않았는데도 모델은 반복해서 학습을 해냈다. 생성자는 얼룩말 처럼 보여 식별자를 속이기 위한 이미지를 생성하는 방법을 학습했으며, 생성된 이미지에서 미심쩍은 구석은 발견할 수 없다(당연한 이야기이지만 식별자는 로테오를 구경해본 적이 없다).

이와 같은 적대적 방식의 훈련이나 이와 유사한 접근 방식을 통해 사람들은 재미있는 생성자를 많이 만들었다. 실제로는 존재하지 않는 진짜 같은 사람 얼굴을 만들거나, 스케치 그림을 실사 이미지나 멋진 풍경 사진으로 변환하기도 했다. 그 밖에도 생성 모델을 통해 사람들은 진짜 같은 소리나 사실적인 글자 혹은 즐길 만한 음악을 만들어냈다. 이런 모델은 앞으로 여러 창작 과정을 돕는 미래 지향적인 도구로 사용될 것이다.



그림 2.8 얼룩말에 올라탄 사람. 얼룩말은 거부하고 있다.

진지하게 말해서, 이런 일들이 미치는 영향이 부풀려졌다고 보기는 어렵다. 우리가 방금 내려받은 도구는 점점 품질이 개선되며 널리 쓰이는 중이다. 그중에서도 얼굴 모습을 서로 바꿔치는 기술은 대중 매체의 주목을 받고 있다. 검색창에 ‘딥페이크^{deep fake}’만 치면 예시를 무더기로 볼 수 있다(인터넷에 있는 모든 것이 그렇듯 업무상 적절하지 않은 것들도 많으니 링크를 클릭할 때는 주의하자).¹

지금까지 이미지를 보고서 새로운 이미지를 만들어 내는 모델을 다뤄봤다. 한 가지 더 기본적인 요소를 둘러보면서 여행을 마무리하자. 바로 자연어 모델이다.

2.3 장면을 설명하는 사전 훈련된 신경망

자연어와 관련된 모델을 직접 체험하기 위해 뤼티엔 루오Ruotian Luo가 만든 이미지 캡션 모델을 사용할 것이다.² 안드레이 카르파티Andrej Karpathy가 설계한 뉴럴토크2NeuralTalk2 모델의 구현체로 보면 된다. 그림 2.9에서 볼 수 있듯이 이 모델은 사진을 보여주면 해당 장면을 설명하는 영어 자막을 만들어낸다. 모델은 ‘얼룩무늬 고양이가 나무 탁자에 기대어 있고, 앞발 하나는 레이저 마우스 위에, 다른 앞발은 검은 노트북 위에 놓여 있다.’ 같은 설명 문장과 사진이 쌍으로 들어 있는 대량의 데이터셋으로 훈련되었다.³

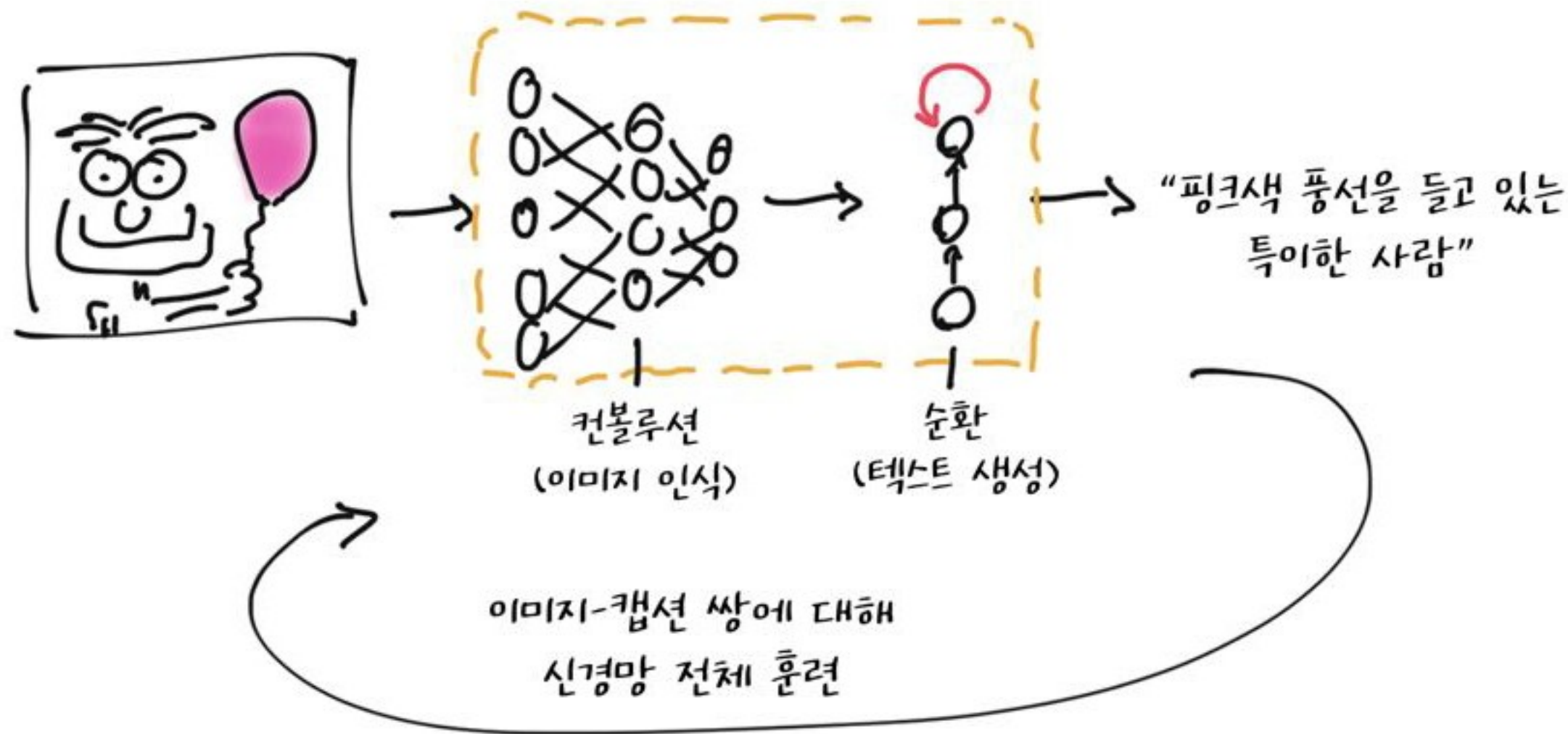


그림 2.9 캡션 모델 개념도

캡션 모델은 크게 반으로 나뉜 신경망이 연결되어 있다. 첫 번째는 장면(얼룩무늬 고양이, 레이저 마우스, 발)을 숫자로 ‘표현하는 법’을 학습하는 신경망으로서, 두 번째 신경망의 입력으로 사용한다. 두 번째는 순환 신경망^{recurrent network}으로 숫자로 기술된 정보로 연관된 문장을 만드는 역할을 한다. 두 모델은 모두 이미지-자막 쌍으로 훈련된다.

두 번째 모델이 ‘순환’ 신경망이라 불리는 이유는 출력을 만드는 방식에서 기인한다. 순방향으로 값이 전달할 때 이전 순방향 전달의 출력값을 함께 넣기 때문이다. 이로 인해 우리가 단어를 연속적으로 나열하며 문장을 만들듯 앞에서 만들어진 단어가 다음 단어에 영향을 주게 된다.

2.3.1 뉴럴토크2

뉴럴토크2는 <https://github.com/deep-learning-with-pytorch/ImageCaptioning.pytorch>에서 찾을 수 있다. data 디렉토리에 이미지 몇 개를 놓고 다음과 같은 스크립트를 실행할 수 있다.

```
python eval.py --model ./data/FC/fc-model.pth --infos_path ./data/FC/fc-infos.pkl \
➡ --image_folder ./data
```

horse.jpg 이미지로 실행하면 ‘해변에서 말을 타고 있는 사람’으로 나오는데 꽤 정확하다.

재미삼아 우리의 사이클GAN이 뉴럴토크2도 속일 수 있는지 확인해보자. 데이터 폴더에 zebra.jpg를 넣고 모델을 돌려보자. 모델은 그림에서 ‘풀밭에 있는 얼룩말 무리’라는 동물 종류는 잘 인식했지만, 그 얼룩말이 두 마리 이상일 거라고 봤다. 아마 신경망은 학습하면서 저런 모습의 얼룩말 사진은 본 적이 없을 것이며, 여러 무리의 얼룩말 사진은 봤어도 사람이 탄 (그럴싸한 패턴의) 얼룩말 사진은 본 적이 없을 것이다. 사실 훈련셋에 여러 무리의 얼룩말이 들어 있을 확률은 매우 크고 그래서 이렇게 편향되게 판단했을 가능성이 있다. 캡션 신경망은 말에 올라탄 ‘사람’은 설명하지 않는데 이 역시 훈련셋에 그런 장면에 있는 사람 데이터가 없었을 가능성이 크다. 어찌되었거나, 인상적인 솜씨로 보인다. 우리는 불가능한 상황을 그린 가짜 이미지를 만들었고 캡션 신경망은 그림의 주제를 제대로 파악할 정도로 충분히 유연하다.

이전에 불가능하다고 생각했던 일들이 오늘날 딥러닝에서 겨우 수천 줄의 코드로 범용적인 아키텍처를 만들어 내서 말이나 얼룩말에 대해 전혀 모르는 상태에서도 이미지와 이를 설명하는 문장(이번에는 MS COCO 데이터셋으로 학습한 것이다) 학습으로 인식이 가능해졌다는 점을 다시 한번 강조하고 싶다. 하드코드된 판단 기준이나 문법은 없으며, 구문을 포함한 모든 것은 데이터 안의 패턴에서 비롯된다. 또는 저절로 생겨난다.

지금 살펴본 신경망 아키텍처는 두 개의 신경망을 포함하는 점에서 이전보다 복잡하다. 순환 신경망이 들어갔지만 이 또한 파이토치에서 제공하는 빌딩 블록으로 만들었다.

이 책을 집필하는 현재 시점에서는, 이런 모델들이 제대로 정의되고 확실한 사용처를 만들어냈 다기보다는 응용 연구나 참신한 시도로서 활용되는 경우가 대부분이다. 결과가 놀랍기는 하지만 아직 실제로 사용될 정도는 아니다. 앞으로 충분한 훈련 데이터와 시간이 주어진다면 이런 모델 유형을 통해 시신경에 손상을 입은 사람들에게 눈앞에 펼쳐진 상황을 설명해주거나 동영상 내용을 기술하는 등의 작업을 제공할 수 있을 것으로 본다.

2.4 토치 허브

딥러닝 초기부터 사람들은 사전 훈련된 모델을 공개해왔다. 하지만 파이토치 1.0 버전까지는, 공개 모델이 동일한 인터페이스를 사용한다는 보장이 없었다. 이 장의 앞부분에서 살펴본 토치 비전은 인터페이스가 깔끔한 경우다. 하지만 사이클GAN과 뉴럴토크2에서 본 것처럼 사람들은 저마다 다른 인터페이스를 디자인하기도 한다.

파이토치는 버전 1.0에서 토치 허브를 소개했다. 제작자가 깃허브에 모델을 공개할 수 있고, 사전 훈련된 가중치의 포함 여부를 선택할 수 있으며, 파이토치가 이해하는 인터페이스를 통해 노출하는 형태였다. 이로 인해 서드 파티로부터 사전 훈련된 모델을 로딩하는 것은 토치비전 모델을 로딩하는 것처럼 쉬워졌다.

토치 허브에 모델을 공개하는 것은 깃허브 저장소의 최상위 디렉토리에 hubconf.py라는 이름의 파일을 올려놓는 것이 전부다. 파일 구조는 다음과 같이 매우 단순하다.

```
dependencies = ['torch', 'math']
```

 ← 코드가 의존하는 모듈 리스트

```
def some_entry_fn(*args, **kwargs):  
    model = build_some_model(*args, **kwargs)  
    return model
```

저장소에 대한 진입점으로 사용자에게
하나 이상의 함수를 노출해야 한다.
이 함수들은 인자값으로 모델을 초기화한 후
모델 인스턴스를 반환해야 한다.

```
def another_entry_fn(*args, **kwargs):  
    model = build_another_model(*args, **kwargs)  
    return model
```


지금까지 살펴본 흥미로운 사전 훈련된 모델 외에, 이제 `hubconf.py`이 들어 있는 깃허브 저장소도 찾아볼 수 있게 됐다. `torch.hub` 모듈을 사용해 `hubconf.py`에 정의된 모듈들을 즉시 로딩할 수 있다. 실제로 어떻게 하는지 연습해보자. 이를 위해 먼저 토치 허브와 어떻게 상호작용하는지 쉽게 이해할 수 있는 예제인 토치비전으로 돌아가자.

`hubconf.py` 파일이 있는 <https://github.com/pytorch/vision> 페이지를 방문하자. 아주 좋다. 파일 내용을 찾아서 나중에 사용할 진입점을 확인하자. 토치 비전의 진입점은 두 곳으로 `resnet18`과 `resnet50`이다. 각각 18개와 50개의 층을 가진 레즈넷 모델을 반환하는 것쯤은 이미 알고 있다. 진입점이 되는 함수에는 `pretrained`라는 키워드 인자가 포함되어 있다. 이 장 앞부분에서 살펴본 것처럼 `True`로 설정하면 모델은 이미지넷으로 사전 훈련된 가중치로 초기화된 채 반환된다.

저장소와 진입점을 둘러봤고, 키워드 인자값도 살펴보았다. 저장소 클론도 필요 없이 torch.hub를 사용해서 모델을 로딩하는 법을 배웠다. 나머지는 파이토치가 알아서 해준다.

```
import torch
from torch import hub
resnet18_model = hub.load('pytorch/vision:master',  ←—— 깃헙 저장소 이름과 브랜치
                        'resnet18',  ←—— 진입점이 되는 함수 이름
                        pretrained=True)  ←—— 키워드 인자
```

이 코드는 pytorch/vision 저장소의 마스터 브랜치 스냅샷을 모델 가중치와 함께 로컬 디렉토리에 다운로드한다(기본 경로로 홈 디렉토리의 .torch/hub을 사용한다). 그리고 resnet18 진입점 함수를 실행하여 모델 인스턴스를 얻는다. 환경에 따라 파이썬이 PIL 같은 모듈이 없다고 메시지를 출력할 수 있다. 토치 허브가 의존성 모듈을 대신 설치해주지는 않지만 메시지로 알려주기 때문에 여기에 따라 대응할 수 있다.

앞서 수행한 작업과 마찬가지로 모델에 적절한 인자를 주어 실행할 수 있다. 훌륭한 점 중 하나는 이제 비전뿐 아니라 모든 모델이 이런 방식으로 접근 가능하다는 점이다.

진입점이 모델을 반환하고 있는 부분에 주의하자. 엄격하게 보자면, 모델을 반환해서는 안 된다. 대신 입력을 다른 형태로 변환하거나 출력 확률을 텍스트 레이블로 바꾸는 역할을 하게 만들 수 있다. 아니면 단순히 모델 자체의 진입점으로 두고, 모델, 전처리, 후처리를 포함한 별개의 루틴을 만들 수도 있다. 이와 같은 다양한 방식이 가능하도록 파이토치 개발자들은 파이토치 커뮤니티에 표준화와 풍부한 유연성을 제공해왔다.

이 책을 쓰는 현재 토치 허브는 다소 생소하며, 이런 식으로 공개되는 모델도 많지 않다. 앞으로 더 많은 사람들이 토치 허브를 통해 모델을 공유하길 바라며, 이로써 구글에서 “github.com hubconf.py”를 검색했을 때 나오는 결과도 더 많아지기를 기대해본다.

2.5 결론

흥미로운 내용이었길 바란다. 우리는 간혹 특정 작업에 최적화된 파이토치 모델을 다루기도 했다. 다른 누군가는 벌써 이런 모델을 웹서버에 올려 사업을 시작하고 거기에서 얻은 수익을 모델 제작자와 나누고 있다는 사실이 정말 놀라웠다!⁴ 모델 구축 방법을 알고 나면, 우리도 여기서 배운 지식을 활용해 사전에 훈련된 모델을 내려받아 빠르게 튜닝하여 조금 다른 작업에도 적용해 볼 수 있다.

또한 다양한 데이터로 다양한 문제를 해결하는 모델 구축이 동일 빌딩 블록을 사용해 어떻게 이뤄지는지 살펴볼 것이다. 파이토치가 정말 잘하는 것 중 하나가 이런 빌딩 블록을 기본 도구 모음으로 제공하는 점이다. API 관점에서 다른 딥러닝 프레임워크와 비교해볼 때, 파이토치는 큰 라이브러리가 아니다.

이 책은 파이토치의 모든 API나 딥러닝의 모든 아키텍처를 살펴보지는 않는다. 대신 이와 같은 빌딩 블록을 실제로 사용하는 경험을 얻게 해준다. 이렇게 튼튼하게 다져진 경험을 바탕으로 훌륭한 온라인 문서와 저장소를 사용할 수 있도록 도와줄 것이다.

다음 3장부터는 이번 2장에서 설명한 내용을 바탕으로, 파이토치를 사용해 컴퓨터에게 기술을 가르쳐줄 수 있도록 바닥부터 살펴볼 것이다. 반대로 바닥부터 시작할 필요 없이 사전 훈련된 신경망을 출발점으로 삼아 새로운 데이터로 훈련시켜서 정교하게 튜닝하는 법도 배울 것이다. 데이터 수가 작을 때 특별히 유용한 방법인데, 이 경험을 통해 딥러닝을 활용하는 관점에서 사전 훈련된 신경망이 유용한 도구인 이유를 알게 될 것이다. 여러분이 가장 먼저 배울 기본 빌딩 블록은 텐서^{tensor}다.

2.7 핵심 요약

- 사전 훈련된 모델은 데이터셋으로 이미 훈련된 모델을 말하며 신경망 파라미터를 로딩한 즉시 의미있는 결과를 만들어낸다.
- 사전 훈련된 모델을 어떻게 사용하는지를 알면 모델을 설계하거나 훈련시킬 필요 없이 바로 신경망을 프로젝트에 활용할 수 있다.
- 알렉스넷과 레즈넷은 발표된 첫 해에 이미지 인식 대회에서 최고 점수를 경신한 심층 컨볼루션 신경망이다.
- GAN, 즉 생성적 적대 신경망은 생성자와 식별자라는 두 부분으로 이뤄져 있으며 실제와 구별하기 어려운 결과를 만들어낸다.
- 사이클GAN은 두 개의 다른 클래스 이미지 간에 서로 변환하는 역할을 한다.
- 뉴럴토크2는 이미지를 입력으로 받고 내용을 설명하는 텍스트를 출력하는 하이브리드 모델 아키텍처를 가진다.
- 토치 허브는 적절한 hubconf.py 파일을 포함한 프로젝트에서 모델과 가중치를 로드하기 위한 표준화된 방식을 제공한다.